

Continental's: Tech Stack & System Requirements Document

1. Technical Stack Specification

The **Continental's Hotel Management System** is structured using a high-performance decoupled architecture. The frontend leverages modern reactive frameworks for fluid UI animations, while the backend utilizes an asynchronous, enterprise-grade architecture backed by a robust relational storage layer.

Layer	Technology Component	Strategic Utility in Continentals
Frontend Core	HTML5, CSS3, JavaScript (ES6+)	Semantic foundation, baseline layouts, and structural DOM modern logic.
Build & Runtime	React + Vite	Component-driven single page app architecture with ultra-fast Hot Module Replacement (HMR).
Styling Framework	Tailwind CSS	Utility-first styles enforcing strict spatial discipline and custom tokenized branding parameters.
Animation Engine	Framer Motion	Orchestrates complex scroll reveal triggers, timeline-based hero curves, and state transitions.
Application Server	Node.js & Express.js	Event-driven, asynchronous RESTful execution environment managing reservation pipelines and room states.
Database Layer	PostgreSQL	Relational engine ensuring ACID compliance for booking transactions, room inventory allocation, and security logs.

2. Dependency Procurement & Setup Guide

Follow these sequential procedures to download and seed the system's foundational dependencies locally. Ensure that `Node.js (v18+)` and `npm` are present on your development system.

Step 2.1: Clone and Isolate Directories

```
# Clone repository and access root directory
cd continentals-system

# Clean install packages across both workspace environments
```

Step 2.2: Frontend Client Dependencies

Navigate into the client directory to initialize the React ecosystem built over Vite:

```
cd client
npm install react react-dom framer-motion lucide-react
npm install -D vite @types/react @types/react-dom tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

Step 2.3: Backend Server Dependencies

Navigate into the server directory to configure your Express environment and PostgreSQL connectors:

```
cd ../server
npm install express pg dotenv cors helmet morgan jsonwebtoken bcryptjs
npm install -D nodemon
```

3. Manual Database Connection Guide

Operational Note: While `Antigravity` natively coordinates automated infrastructure setups, container configurations, and hot-linking pipelines, manual intervention parameters are preserved below for localized diagnostic debugging.

Step 3.1: PostgreSQL Instance Provisioning

Log in to your localized PostgreSQL terminal (psql) or administration dashboard (pgAdmin) and execute the structural database initialization queries:

```
CREATE DATABASE continentals_db;
CREATE USER continentals_admin WITH PASSWORD 'SecurePass99!';
GRANT ALL PRIVILEGES ON DATABASE continentals_db TO continentals_admin;
```

Step 3.2: Configuration of Environment Credentials

Create a `.env` file directly under the `/server` directory structure:

```

PORT=5000
DB_USER=continentals_admin
DB_HOST=localhost
DB_NAME=continentals_db
DB_PASSWORD=SecurePass99!
DB_PORT=5432
JWT_SECRET=premium_continental_token_system

```

Step 3.3: Manual Pool Connection Script

The localized initialization connection is governed via Node-Postgres (pg). Inside `/server/config/database.js`:

```

const { Pool } = require('pg');
require('dotenv').config();

const pool = new Pool({
  user: process.env.DB_USER,
  host: process.env.DB_HOST,
  database: process.env.DB_NAME,
  password: process.env.DB_PASSWORD,
  port: process.env.DB_PORT,
});

pool.on('connect', () => {
  console.log('Database connected successfully via manual pool protocol.');
```

```

});

module.exports = pool;

```

4. Architectural Codebase Comprehension Guide

The source layout splits responsibilities strictly down structural layers, enabling highly professional decoupled separation of concerns.

```

continentals-system/
├── client/                                # Frontend client runtime built on React + Vite
│   ├── src/
│   │   ├── components/                  # Global shared modules (Premium Navigation, Buttons)
│   │   ├── pages/                      # Primary route view states
│   │   │   ├── GuestHome.jsx           # Premium landing view containing custom Framer Scroll Reveals
│   │   │   └── AdminPortal.jsx         # Clean data table dashboard containing room layout columns
│   │   ├── styles/                    # Global Tailwind stylesheets and token rules
│   │   ├── App.jsx                    # React structural application router
│   │   └── main.jsx                   # DOM compiler mount point
│   ├── tailwind.config.js             # Custom theme token configurations (#0F172A, #2563EB)
│   └── vite.config.js                 # Optimization compilation rules
└── server/                            # Backend microservices layer built on Node + Express
    ├── config/                       # Static asset parameters & direct DB connection keys
    ├── controllers/                  # Operational lifecycle logic for transactions & inventory
    ├── routes/                      # Explicit REST endpoints mapped cleanly to HTTP operations
    └── server.js                     # Primary entry point initialization file

```

5. Run Execution Strategy (Antigravity Orchestration)

Under execution runtime environments, **Antigravity** automates the orchestration framework, injecting variables and configuring database networking layers instantly.

The Automagic Orchestration Routine

Execute the central activation hook from the parent root folder path:

```
antigravity up --mode development
```

Upon triggering, the automation engine instantly parses the file system, coordinates localized port forwarding (Frontend to `:5173`, Backend to `:5000`), automatically spins up localized PostgreSQL runtime pools, runs raw structural table migrations, and surfaces the platform without manual script adjustments.

Manual Fallback Execution Protocol

Should you choose to operate outside the automatic engine workspace, initialize the environments concurrently via divided separate terminals:

```
# Terminal 1: Trigger Backend Ecosystem
cd server
npm run dev (or nodemon server.js)

# Terminal 2: Trigger Frontend Client
cd client
npm run dev
```